Tokenization using Lex Tool

Program (tokenizer.l)

```
%{
#include <stdio.h>
int count = 0;
%}

%%

"int"|"float"|"char"|"double"|"void"|"if"|"else"|"while"|"for"|"return" {
    printf("<KEYWORD, %s>\n", yytext); count++;
}

[a-zA-Z_][a-zA-Z0-9_]* {
    printf("<IDENTIFIER, %s>\n", yytext); count++;
}

[0-9]+(\.[0-9]+)? {
    printf("<NUMBER, %s>\n", yytext); count++;
}

"=="|"!="|"<="|">="|"&&"|"||"|"+"|"-"|"*"|"/"|"%"|"="|"<"|">"|"!" {
    printf("<OPERATOR, %s>\n", yytext); count++;
}

[(){}\[\];,] {
    printf("<SEPARATOR, %s>\n", yytext); count++;
}

[ \t\n]+    ;

. {
    printf("<UNKNOWN, %s>\n", yytext); count++;
}

%%

int yywrap() { return 1; }

int main() {
    printf("Enter input (Ctrl+D to end):\n");
    yylex();
    printf("\nTotal tokens: %d\n", count);
    return 0;
}
```

How to run

```
lex tokenizer.l
gcc lex.yy.c -o tokenizer
./tokenizer
```

Sample input

```
int a = 10 + b;
```

Sample output

```
<KEYWORD, int>
<IDENTIFIER, a>
```

```
<OPERATOR, =>
<NUMBER, 10>
<OPERATOR, +>
<IDENTIFIER, b>
<SEPARATOR, ;>

Total tokens: 7
```